

```

//
=====
==
// PATTERN: HOSPITAL APPOINTMENT SYSTEM (Transaction/Booking System)
//
=====
==
// This is like Pattern 1 (Concert Tickets) but for a HOSPITAL
// Use this when you see: bookings, appointments, medical records, patient
care
//
=====
==

//
=====
==
// FILE 1: Specialization.java (ENUM - separate file)
//
=====
==
// Right-click → New → Enum
// If CONCERT has Genre → HOSPITAL has Specialization
// If CAR DEALER has CarType → HOSPITAL has Specialization

enum Specialization {
    CARDIOLOGY,           // Heart specialist
    DERMATOLOGY,          // Skin specialist
    NEUROLOGY,            // Brain specialist
    ORTHOPEDICS,          // Bone specialist
    PEDIATRICS,           // Children specialist
    GENERAL_PRACTICE      // General doctor

    // EXAM TIP: Replace these with the specialties from YOUR exam spec
    // Could be: departments, categories, types, etc.
}

//
=====
==
// FILE 2: FacilityType.java (ENUM - separate file)
//
=====
==
// Right-click → New → Enum
// If CONCERT has VenueType → HOSPITAL has FacilityType
// If CAR DEALER has ShowroomType → HOSPITAL has FacilityType

```

```

enum FacilityType {
    PUBLIC_HOSPITAL,
    PRIVATE_HOSPITAL,
    CLINIC,
    HEALTH_CENTRE,
    SPECIALIST_CENTRE,
    NOT_SPECIFIED

    // EXAM TIP: Replace with facility/location types from YOUR exam
}

//
=====
==
// FILE 3: Region.java (RECORD - separate file)
//
=====
==
// Right-click → New → Record
// If CONCERT has Country → HOSPITAL has Region
// If CAR DEALER has Location → HOSPITAL has Region

import java.util.*;

record Region(
    String name,
    double averageWaitTime,      // Average wait time in hours
    int populationDensity,      // People per sq km
    boolean hasEmergencyServices
) {
    // REPLACE: name with region/location name from exam
    // REPLACE: averageWaitTime with relevant metric (distance, rating,
    etc.)
    // REPLACE: populationDensity with relevant metric
    // REPLACE: hasEmergencyServices with relevant boolean

    // Validation in compact constructor (DOP Principle 3)
    public Region {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Region name is required");
        }
        if (averageWaitTime < 0) {
            throw new IllegalArgumentException("Wait time cannot be
negative");
        }
    }
}

```

```

}

// Pre-defined regions (like CountryRegistry in concert example)
class RegionRegistry {
    public static final Region DUBLIN =
        new Region("Dublin", 2.5, 1400, true);
    public static final Region CORK =
        new Region("Cork", 3.0, 800, true);
    public static final Region GALWAY =
        new Region("Galway", 4.0, 500, false);

    // EXAM TIP: Create constants for regions mentioned in exam spec
}

//
=====
==
// FILE 4: Doctor.java (RECORD - separate file)
//
=====
==
// Right-click → New → Record
// If CONCERT has Artist → HOSPITAL has Doctor
// If CAR DEALER has Salesperson → HOSPITAL has Doctor

record Doctor(
    String id,
    String name,
    Specialization specialization    // Their specialty
) {
    // REPLACE: Doctor with main entity (Teacher, Agent, Consultant, etc.)
    // REPLACE: specialization with relevant category from exam

    // Constructor that auto-generates ID
    public Doctor(String name, Specialization specialization) {
        this(UUID.randomUUID().toString(), name, specialization);
    }

    public Doctor {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Doctor name is required");
        }
    }
}

```

```

//
=====
==
// FILE 5: Facility.java (RECORD - separate file)
//
=====
==
// Right-click → New → Record
// If CONCERT has Venue → HOSPITAL has Facility
// If CAR DEALER has Dealership → HOSPITAL has Facility

record Facility(
    String id,
    String name,
    FacilityType type,
    Region region,
    int capacity                // Max patients per day
) {
    // REPLACE: Facility with location entity from exam
    // REPLACE: capacity with relevant number (seats, spaces, etc.)

    public Facility(String name, FacilityType type, Region region, int
capacity) {
        this(UUID.randomUUID().toString(), name, type, region, capacity);
    }

    public Facility {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Facility name is
required");
        }
        if (capacity < 10) { // REPLACE 10 with minimum from exam rules
            throw new IllegalArgumentException("Capacity must be at least
10");
        }
    }
}

//
=====
==
// FILE 6: Appointment.java (RECORD - separate file)
//
=====
==
// Right-click → New → Record
// If CONCERT has Concert → HOSPITAL has Appointment

```

```

// If CAR DEALER has TestDrive → HOSPITAL has Appointment

import java.time.LocalDateTime;

record Appointment(
    String id,
    Doctor doctor,           // The doctor
    LocalDateTime dateTime,  // When
    Facility facility        // Where
) {
    // REPLACE: Appointment with event entity from exam
    // REPLACE: doctor with main performer/provider
    // REPLACE: facility with location

    public Appointment(Doctor doctor, LocalDateTime dateTime, Facility
facility) {
        this(UUID.randomUUID().toString(), doctor, dateTime, facility);
    }

    public Appointment {
        // Rule: Appointment must be in future
        if (dateTime.isBefore(LocalDateTime.now())) {
            throw new IllegalArgumentException("Appointment must be in
future");
        }
        // REPLACE: Add rules from YOUR exam spec
    }
}

//
=====
==
// FILE 7: Patient.java (RECORD - separate file)
//
=====
==
// Right-click → New → Record
// If CONCERT has Attendee → HOSPITAL has Patient
// If CAR DEALER has Customer → HOSPITAL has Patient

import java.util.*;

record Patient(
    String id,
    String name,
    String email,
    List<Booking> bookings    // Their bookings

```

```

) {
    // REPLACE: Patient with user/customer entity from exam
    // REPLACE: bookings with relevant collection (purchases,
registrations, etc.)

    public Patient(String name, String email, List<Booking> bookings) {
        this(UUID.randomUUID().toString(), name, email,
            Collections.unmodifiableList(new ArrayList<>(bookings)));
    }

    public Patient {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Valid email required");
        }
        // Make list immutable (DOP Principle 2)
        bookings = Collections.unmodifiableList(new
ArrayList<>(bookings));
    }
}

//
=====
==
// FILE 8: PatientAdvisory.java (SEALED INTERFACE - separate file)
//
=====
==
// Right-click → New → Interface (then add "sealed" manually)
// If CONCERT has Advisory → HOSPITAL has PatientAdvisory
// If CAR DEALER has PurchaseWarning → HOSPITAL has PatientAdvisory

// This is ONE file containing the sealed interface AND all nested records
sealed interface PatientAdvisory permits
    PatientAdvisory.BringInsurance,
    PatientAdvisory.FacilityBusy,
    PatientAdvisory.LongWaitTime,
    PatientAdvisory.NoEmergencyServices {

    String message();

    // NESTED RECORD 1 (inside this same file!)
    record BringInsurance() implements PatientAdvisory {
        public String message() {
            return "Remember to bring health insurance card";
        }
    }
}

```

```

// NESTED RECORD 2 (inside this same file!)
record FacilityBusy() implements PatientAdvisory {
    public String message() {
        return "Warning: Large facility may have longer wait times";
    }
}

// NESTED RECORD 3 (inside this same file!)
record LongWaitTime() implements PatientAdvisory {
    public String message() {
        return "Expected wait time exceeds 3 hours in this region";
    }
}

// NESTED RECORD 4 (inside this same file!)
record NoEmergencyServices() implements PatientAdvisory {
    public String message() {
        return "This region has no emergency services - plan
accordingly";
    }
}

// EXAM TIP: Replace these with advisories/warnings from YOUR exam
rules
// All these records stay INSIDE this one PatientAdvisory.java file
}

//
=====
==
// FILE 9: Booking.java (RECORD - separate file)
//
=====
==
// Right-click → New → Record
// If CONCERT has Ticket → HOSPITAL has Booking
// If CAR DEALER has Sale → HOSPITAL has Booking

record Booking(
    String id,
    Appointment appointment,
    Patient patient,
    double fee, // Cost of appointment
    List<PatientAdvisory> advisories // Computed warnings
) {
    // REPLACE: Booking with transaction entity from exam
    // REPLACE: fee with price/cost field

```

```

// REPLACE: advisories with computed notifications

public Booking(Appointment appointment, Patient patient, double fee) {
    this(UUID.randomUUID().toString(),
        appointment, patient, fee,
        computeAdvisories(appointment));
}

public Booking {
    if (fee < 0) { // Rule: Fee cannot be negative
        throw new IllegalArgumentException("Fee cannot be negative");
    }
    advisories = Collections.unmodifiableList(new
ArrayList<>(advisories));
}

// COMPUTE ADVISORIES based on rules (THIS IS KEY!)
private static List<PatientAdvisory> computeAdvisories(Appointment
appointment) {
    List<PatientAdvisory> advs = new ArrayList<>();
    Region region = appointment.facility().region();
    Facility facility = appointment.facility();

    // Rule 1: Check if private hospital (REPLACE with YOUR rule)
    if (facility.type() == FacilityType.PRIVATE_HOSPITAL) {
        advs.add(new PatientAdvisory.BringInsurance());
    }

    // Rule 2: Check facility capacity (REPLACE with YOUR rule)
    if (facility.capacity() > 100) {
        advs.add(new PatientAdvisory.FacilityBusy());
    }

    // Rule 3: Check wait time (REPLACE with YOUR rule)
    if (region.averageWaitTime() > 3.0) {
        advs.add(new PatientAdvisory.LongWaitTime());
    }

    // Rule 4: Check emergency services (REPLACE with YOUR rule)
    if (!region.hasEmergencyServices()) {
        advs.add(new PatientAdvisory.NoEmergencyServices());
    }

    return advs;
}
}

```



```

//
=====
==
// FILE 10: AppointmentOperation.java (SEALED INTERFACE - separate file)
//
=====
==
// Right-click → New → Interface (then add "sealed" manually)
// If CONCERT has TicketOperation → HOSPITAL has AppointmentOperation
// If CAR DEALER has SaleOperation → HOSPITAL has AppointmentOperation

import java.util.function.Predicate;

// This is ONE file containing sealed interface AND nested records
sealed interface AppointmentOperation<T> permits
    AppointmentOperation.Book,
    AppointmentOperation.Search {

    // NESTED RECORD 1: Book operation (inside this same file!)
    record Book(
        Patient patient,
        Appointment appointment,
        double fee
    ) implements AppointmentOperation<Booking> {}
    // REPLACE: Book with main action (Purchase, Register, Apply, etc.)

    // NESTED RECORD 2: Search operation (inside this same file!)
    record Search(
        List<Appointment> appointments,
        Predicate<Appointment> criteria
    ) implements AppointmentOperation<List<Appointment>> {}
    // REPLACE: criteria logic with YOUR search requirements
}

//
=====
==
// FILE 11: HospitalSystem.java (CLASS - separate file)
//
=====
==
// Right-click → New → Class
// If CONCERT has TicketSystem → HOSPITAL has HospitalSystem
// If CAR DEALER has DealershipSystem → HOSPITAL has HospitalSystem

class HospitalSystem {

```

```

        // OPERATION 1: Book appointment (FACTORY PATTERN)
        public static Booking bookAppointment(AppointmentOperation.Book op) {
            // Create new booking
            Booking booking = new Booking(op.appointment(), op.patient(),
op.fee());

            // Update patient with new booking (immutability requires
reconstruction)
            List<Booking> updatedBookings = new
ArrayList<>(op.patient().bookings());
            updatedBookings.add(booking);
            Patient updatedPatient = new Patient(
                op.patient().name(),
                op.patient().email(),
                updatedBookings
            );

            System.out.println("Appointment booked for: " +
updatedPatient.name());
            return booking;
        }

        // OPERATION 2: Search appointments with filter
        public static List<Appointment>
searchAppointments(AppointmentOperation.Search op) {
            return op.appointments().stream()
                .filter(op.criteria())
                .toList();
        }

        // OPERATION 3: Get advisories for appointment
        public static List<PatientAdvisory> getAdvisories(Appointment
appointment) {
            Booking tempBooking = new Booking(appointment, null, 0.0);
            return tempBooking.advisories();
        }

        // EXAM TIP: Add more operations based on YOUR exam requirements
        // Examples: cancel(), reschedule(), getByDoctor(), etc.
    }

//
=====
==
// FILE 12: Runner.java (CLASS - separate file)

```

```
//
=====
==
// Right-click → New → Class (check "public static void main")

import java.time.LocalDateTime;
import java.time.Month;

class Runner {
    public static void main(String[] args) {
        // Step 1: Create entities (REPLACE names with exam data)
        var doctor = new Doctor("Dr. Sarah Smith",
Specialization.CARDIOLOGY);
        var facility = new Facility("St. James Hospital",
FacilityType.PUBLIC_HOSPITAL,
                                RegionRegistry.DUBLIN, 150);
        var appointment = new Appointment(doctor,
                                LocalDateTime.of(2025, 12, 20,
14, 30),
                                facility);
        var patient = new Patient("John Murphy", "john.murphy@email.com",
List.of());

        // Step 2: Book appointment
        var booking = HospitalSystem.bookAppointment(
            new AppointmentOperation.Book(patient, appointment, 50.00)
        );
        System.out.println("Booking created: " + booking);

        // Step 3: Search appointments (REPLACE criteria with exam
requirement)
        List<Appointment> results = HospitalSystem.searchAppointments(
            new AppointmentOperation.Search(
                List.of(appointment),
                apt -> apt.dateTime().isBefore(LocalDateTime.of(2025, 12,
25, 0, 0))
            )
        );
        System.out.println("Search results: " + results);

        // Step 4: Update patient (immutability pattern - must recreate)
        var updatedBookings = new ArrayList<>(patient.bookings());
        updatedBookings.add(booking);
        patient = new Patient(patient.name(), patient.email(),
updatedBookings);

        // Step 5: Check advisories
```

```

        System.out.println("Patient advisories: " + booking.advisories());
        for (PatientAdvisory advisory : booking.advisories()) {
            System.out.println("- " + advisory.message());
        }
    }
}

//
=====
==
// EXAM REPLACEMENT GUIDE - HOSPITAL vs YOUR EXAM
//
=====
==
// HOSPITAL SYSTEM          → YOUR EXAM SYSTEM
//
=====
==
// Doctor                    → Main entity (Teacher, Agent, Performer)
// Patient                    → User entity (Student, Customer, Attendee)
// Appointment                → Event (Class, Meeting, Viewing)
// Booking                    → Transaction (Enrollment, Purchase,
Reservation)
// Facility                    → Location (School, Office, Venue)
// Region                     → Geography (District, Zone, Area)
// Specialization              → Category (Subject, Type, Genre)
// FacilityType                → Location type
// PatientAdvisory             → Warnings/notifications
// HospitalSystem              → MainSystem
// AppointmentOperation        → MainOperation
//
=====
==

```